ReliableML Method Note: Meta-Learning Classifiers

Jabulani Monchwe, Olorato Motitswane, and Mfundo Monchwe

Version 0.1.1

Abstract

The meta-learning estimators transform predictions from a configurable pool of base classifiers into training features for a second-stage classifier. Cross-fitting supplies out-of-fold predictions during training to reduce target leakage from in-sample base-model outputs.

1 Stacking procedure

Given training data (X, y), stratified folds are created. In each fold, base candidates are fit on fold-training rows and produce aligned class probabilities for held-out rows. These out-of-fold outputs are joined as meta-features. The second-stage classifier is trained on those features and labels. At inference, fitted base learners produce the same feature layout, which is passed to the fitted meta-classifier. The user supplies base estimators; they can be ordinary scikit-learn classifiers.

2 Implemented estimators

`PerfMetaSelect` Selects the candidate with the best mean stratified cross-validation score, then refits it on all training data.
`MetaFeatureTransformer` Exposes class-aligned out-of-fold probability features for use in a scikit-learn workflow.
`MetaPipelineClassifier` Fits the OOF meta-feature procedure with a configurable final classifier.
`CalibratedStackedClassifier` Stacks OOF probabilities and calibrates a second-stage estimator using sigmoid or isotonic calibration.
`StatisticalMetaClassifier` Builds statistical meta-features and fits a logistic statistical model with selectable estimation and nonlinear terms.
`StatisticalMetaLDAClassifier` Linear statistical variant using discriminant projections and logistic estimation.
`StatisticalMetaDTreeClassifier` Nonlinear statistical variant with optional tree meta-features and polynomial interactions.
`StatisticalMetaNNClassifier` Neural meta-network over LDA, optional tree, and cluster-distance features; optional Monte Carlo dropout estimates stochastic prediction variation.
Raw-feature importance methods are provided where supported by the estimator. The reported aggregate importance across base models is a descriptive frequency/importance summary, not causal importance.

3 Hyperparameter search and validation

`halving_grid_search` and `halving_random_search` wrap scikit-learn successive-halving search. Data-dependent preprocessing should be inside each candidate's estimator pipeline so it is refit within folds. Choose a scoring rule consistent with the task and use nested or otherwise separated validation when reporting final generalization performance.

4 Constraints

Statistical meta-classifiers and the neural statistical estimator currently support binary classification; general stacking wrappers may handle multiple classes when base learners support them. Inputs are finite numeric features. Fold counts must be appropriate for the smallest class. Cross-fitting reduces, but does not eliminate, selection bias; repeated tuning on the same holdout invalidates its role as an independent test. Persist the complete fitted pipeline so inference reproduces the meta-feature transform.

5 Interface

```
meta = MetaPipelineClassifier(
    base_estimators=candidates,
    meta_classifier=LogisticRegression(max_iter=500),
    n_folds=5,
)
meta.fit(X_train, y_train)
predictions = meta.predict(X_test)
```

6 Further reading

References

[1] D. H. Wolpert. Stacked generalization. *Neural Networks*, 5(2), 1992. https://doi.org/10.1016/S0893-6080(05)80023-1.

[2] ReliableML estimators and configuration: https://github.com/Jabualani/ML-Reliabilty.